9.1.3　Quartz 任务调度

1. Quartz 概述

Quartz 是基于 Java 语言实现的开源的任务调度库，它可以集成于 J2EE 或 J2SE 应用程序中，也可以单独使用。Quartz 可以灵活地实现各种任务的调度，支持任务和调度的多种组合方式，并支持调度数据的多种存储方式。

Quartz 采用基于多线程的架构，其内部包含很多重要的组件，下面对这些组件进行说明。

- Job：Job 为希望被调度程序执行的任务，需要设置唯一标识，以便调度的时候进行识别。
- JobDetail：Quartz 并不存储 Job 类的实际实例，而是允许使用 JobDetail 来定义 Job 类的实例。JobDetail 可以设置关联 Job 的详细信息，JobDetail 实例通过 JobBuilder 类创建。
- Trigger：Trigger 为触发器，用来触发执行 Job，通过调整触发器的属性设置 Job 执行的条件，指定任务在什么时候会执行。多个触发器可以指向同一个任务，但单个触发器只能指向一个任务。
- Scheduler：Scheduler 为调度器，调度器会将任务和触发器整合起来，负责基于触发器设定的时间规则来执行 Job。

2. Spring Boot 整合 Quartz

Quartz 作为一款重要的任务调度框架，在使用 Quartz 进行开发和实现应用程序的过程中，需要秉承严谨科学的工作态度，分析任务调度和执行的科学性，以及思考程序如何为用户提供更好的服务和体验，而不是利用用户的隐私和信息谋取不当利益。需要防止任务调度和执行过程中出现的不当行为，并减少资源的浪费，为信息化社会的发展做出积极的贡献。

下面通过一个 Spring Boot 整合 Quartz 的案例演示使用 Quartz 实现任务调度 具体如下。

（1）添加依赖

在 Spring Boot 2.0 之后，Spring Boot 整合了 Quartz，提供了 Quartz 的启动器依赖，在项目 chapter09 的 pom.xml 文件中引入 Quartz 的启动器依赖，具体如文件 9-6 所示。

文件 9-6　pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-quartz</artifactId>
</dependency>
```

（2）创建任务类

任务类需要实现 Job 接口或者继承 Job 的实现类。Spring 提供了一个 Job 的实现类 QuartzJobBean，在 Spring Boot 项目中编写 Job 任务类时，只需要继承 QuartzJobBean 类，并在重写该类的 executeInternal()方法中编写任务的逻辑代码。在 chapter09 项目中创建名称为 com.itheima.chapter09.task 的包，并在包下创建任务类，具体如文件 9-7 所示。

文件 9-7　MyTask.java

```
1  import org.quartz.JobExecutionContext;
2  import org.springframework.scheduling.quartz.QuartzJobBean;
3  import java.time.LocalDateTime;
4  import java.time.format.DateTimeFormatter;
```

```
 5 public class MyTask extends QuartzJobBean {
 6     DateTimeFormatter fm = DateTimeFormatter.ofPattern("yyyy 年 MM 月 dd 日 HH:mm:
ss");
 7     @Override
 8     public void executeInternal(JobExecutionContext context)  {
 9         System.out.println("MyTask 执行，当前时间为："+LocalDateTime.now().format
(fm));
10     }
11 }
```

在上述代码中，自定义的任务类 MyTask 继承了 QuartzJobBean 类，并在重写的 executeInternal()方法中输出每次任务执行时的时间。

（3）创建配置类

任务创建后，需要使用调度器根据触发器中指定的规则执行任务，Spring Boot 和 Quartz 整合后只需将创建执行对应任务的调度器以及对应的触发器交由 Spring 管理即可。在 chapter09 项目中创建名称为 com.itheima.chapter09.config 的包，并在包下创建 Quartz 的配置类，在该配置类中创建任务实例、触发任务执行的触发器，以及调度任务和触发器的调度器，具体如文件 9-8 所示。

文件 9-8 QuartzConfig.java

```
 1 import com.itheima.chapter09.task.MyTask;
 2 import org.quartz.*;
 3 import org.springframework.context.annotation.Bean;
 4 import org.springframework.context.annotation.Configuration;
 5 @Configuration
 6 public class QuartzConfig {
 7     @Bean
 8     public JobDetail jobDetail(){
 9         //构建 Job 信息
10         return JobBuilder.newJob(MyTask.class)
11                 .withIdentity("MyTask")         //使用一个唯一标识作为 JobDetail 的名称
12                 .storeDurably()                 //任务执行完之后进行保留
13                 .build();                       //创建 JobDetail 实例
14     }
15     @Bean
16     public Trigger myTaskTrigger(){
17         //创建基于 Cron 表达式的调度器构建器
18         CronScheduleBuilder schedule = CronScheduleBuilder.cronSchedule("* * *
* * ?");
19         return TriggerBuilder.newTrigger()    //创建 TriggerBuilder 对象
20                 .forJob(jobDetail())          //设置要触发的任务
21                 .withIdentity("MyTaskTrigger")//设置触发器的唯一标识
22                 .withSchedule(schedule)       //设置触发器使用的调度器构建器
23                 .build();                     //创建对应的触发器实例
24     }
25 }
```

在上述代码中，第 7～14 行代码中的 JobDetail()方法根据设置的任务信息，生成对应的 JobDetail 实例并交由 Spring 管理。其中，第 10 行代码指定创建一个 JobBuilder 实例，并设置要执行的 Job 的 class 名称；第 11 行代码使用一个唯一标识作为 JobDetail 实例的名称，

如果没有进行设置，则会生成一个随机的唯一 JobKey 作为 JobDetail 的名称；第 12 行代码设置当没有触发器指向任务时，保留该任务。

第 15～24 行代码的 myTaskTrigger()方法根据对应的触发器信息和调度器构建器信息，生成对应的 Trigger 实例并交由 Spring 管理。其中，第 18 行代码创建的 CronScheduleBuilder 为 ScheduleBuilder 的子类，CronScheduleBuilder 可以创建一个基于 Cron 表达式的触发器，CronScheduleBuilder 通过 cronSchedule()方法指定对应的 Cron 表达式；第 19 行代码创建 TriggerBuilder 对象，使用该对象可以定义触发器的规范；第 20 行代码设置当前触发器触发后执行的任务；第 22 行代码设置触发器使用的调度器构建器。

（4）测试效果

为了使测试结果只展示与 Quartz 任务调度相关的结果，将项目 chapter09 启动类上@EnableAsync、@EnableScheduling 注解注释掉，以关闭之前使用基于 Spring 实现的异步任务和定时任务。启动项目 chapter09，测试 Quartz 任务调度的效果，此时 IDEA 控制台输出信息如图 9-6 所示。

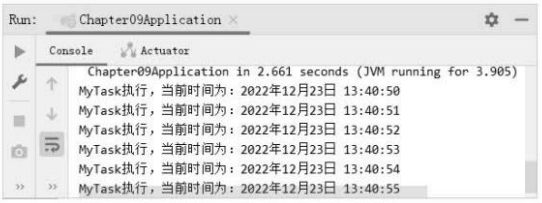


图9-6　Quartz任务调度的效果

从图 9-6 可以看出，每隔一秒输出一次信息，说明调度器根据触发器中定义的 Cron 表达式每隔一秒执行了一次任务，实现了任务的调度。

9.2　邮件发送

发送电子邮件是很多应用程序的常见需求，例如使用电子邮件实现用户注册验证、密码重置、给用户发送营销信息等功能。作为一名程序员，在实现邮件发送时，需要严格遵守职业道德准则，以及相关的规范和法律法规，确保发送邮件的目的是合法的，并且确保邮件的内容不会侵犯他人的隐私权，同时应遵循法律法规对邮件的内容进行严格的审核。

Spring 提供了一个实用的发送电子邮件库，它为使用者屏蔽了邮件系统的底层细节和客户端的底层资源处理，Spring Boot 框架对 Spring 提出的邮件服务也进行了整合支持。下面对在 Spring Boot 项目中实现多种形式的邮件发送进行讲解。

9.2.1　发送纯文本邮件

在邮件发送任务中，最简单的实现莫过于纯文本邮件的发送。在定制纯文本邮件时，只需要指定收件人邮箱账号、邮件标题和邮件内容。下面通过案例演示 Spring Boot 项目中实现纯文本邮件的发送，具体如下。

1. 添加依赖

Spring Boot 为邮件服务提供了对应的启动器，在项目 chapter09 的 pom.xml 文件中，添加 Spring Boot 整合邮件服务的启动器依赖，具体代码如下。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

添加上述依赖后，启动项目后 Spring Boot 整合邮件服务的自动配置就会生效，在邮件发送任务时，可以直接使用 Spring 框架提供的 JavaMailSender 接口或者其实现类 JavaMailSenderImpl 实现邮件发送。

2. 设置邮件服务配置

为了确保邮件服务正常发送，需要在项目的配置文件中设置邮件服务相关的配置。在项目 chapter09 的 application.properties 配置文件中添加发件人邮箱服务器配置和邮件服务超时的相关配置，具体如文件 9-9 所示。

文件 9-9 application.properties

```
# 发件人邮箱服务器相关配置
spring.mail.host=smtp.qq.com
spring.mail.port=587
# 配置个人 QQ 账户和密码（密码是加密后的授权码）
spring.mail.username=461565942@qq.com
spring.mail.password=ijdjokspcbnzfbfa
spring.mail.default-encoding=UTF-8
# 邮件服务超时时间配置
spring.mail.properties.mail.smtp.connectiontimeout=5000
spring.mail.properties.mail.smtp.timeout=3000
spring.mail.properties.mail.smtp.writetimeout=5000
```

在文件 9-9 中，主要添加了发件人邮箱服务器配置和邮件服务超时配置这两部分内容。其中，发件人邮箱服务器配置中，必须明确发件人邮箱对应的 host（服务器主机）、port（端口号）以及用于发件人认证的 username（邮箱账号）和 password（密码）。邮件服务超时配置可以灵活更改超时时间，如果没有配置邮件服务超时的话，Spring Boot 内部默认超时是无限制的，这可能会造成线程被无响应的邮件服务器长时间阻塞。

需要说明的是，本案例中配置的发件人邮箱是 QQ 邮箱，如果读者配置的是其他邮箱，必须更改对应的服务器主机、端口号；另外，配置的密码不是 QQ 邮箱的原始密码，而是通过手机短信验证后的授权码。授权码是 QQ 邮箱推出的用于登录第三方客户端的专用密码。适用于登录 POP3/IMAP/SMTP/Exchange/CardDAV/CalDAV 服务，开启对应的服务时可以获取对应的授权。登录 QQ 邮箱后单击“设置”→“邮箱设置”→“账号”，在账号设置中找到服务设置项，如图 9-7 所示。

在图 9-7 中单击“开启服务”按钮后，会弹出验证密保的对话框，如图 9-8 所示。



图9-7 开启服务



图9-8 验证密保

根据图 9-8 提示的内容进行验证后，会弹出对应的服务的授权码。

3. 定制邮件发送服务

在 com.itheima.chapter09.service 的包中 ,新建一个邮件发送任务管理的业务处理类 SendEmailService，并在该类中编写一个发送纯文本邮件的业务方法，具体如文件 9-10 所示。

文件 9-10　SendEmailService.java

```
1  import org.springframework.beans.factory.annotation.Autowired;
2  import org.springframework.beans.factory.annotation.Value;
3  import org.springframework.mail.MailException;
4  import org.springframework.mail.SimpleMailMessage;
5  import org.springframework.mail.javamail.JavaMailSenderImpl;
6  import org.springframework.stereotype.Service;
7  @Service
8  public class SendEmailService {
9      @Autowired
10     private JavaMailSenderImpl mailSender;
11     @Value("${spring.mail.username}")
12     private String from;
13     public void sendSimpleEmail(String to, String subject, String text) {
14         // 定制纯文本邮件信息
15         SimpleMailMessage message = new SimpleMailMessage();
16         message.setFrom(from);
17         message.setTo(to);
18         message.setSubject(subject);
19         message.setText(text);
20         try {
21             // 发送邮件
22             mailSender.send(message);
23             System.out.println("纯文本邮件发送成功");
24         } catch (MailException e) {
25             System.out.println("纯文本邮件发送失败 " + e.getMessage());
26             e.printStackTrace();
27         }
28     }
29 }
```

在上述代码中，第 9～10 行代码注入了一个 JavaMailSenderImpl 对象，可以通过该对象完成邮件发送。第 11～12 行代码读取配置文件中 spring.mail.username 的值赋给变量 from。第 13～28 行代码定义 sendSimpleEmail()方法用于发送纯文本邮件。其中，第 15～19 行代码创建 SimpleMailMessages 普通邮件模板对象 ,并为该模板对象设置邮件发送的信息 ;第 16～19 行代码依次设置邮件信息的发件人地址、收件人地址、邮件标题和邮件正文；第 22 行代码将设置好信息的模板对象传入 send()方法，执行邮件发送。

4. 定义测试方法

在项目 chapter09 测试类 Chapter09ApplicationTests 中定义方法测试纯文本邮件发送 ,具体如文件 9-11 所示。

文件 9-11　Chapter09ApplicationTests.java

```
1  import com.itheima.chapter09.service.SendEmailService;
2  import org.junit.jupiter.api.Test;
```

```
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.boot.test.context.SpringBootTest;
5  @SpringBootTest
6  class Chapter09ApplicationTests {
7      @Autowired
8      private SendEmailService sendEmailService;
9      @Test
10     public void sendSimpleMailTest() {
11         String to="1175559518@qq.com";
12         String subject="【纯文本邮件测试】";
13         String text="Spring Boot 纯文本邮件发送内容测试……";
14         // 调用邮件发送方法
15         sendEmailService.sendSimpleEmail(to,subject,text);
16     }
17 }
```

在上述代码中，第 7～8 行代码注入 SendEmailService 对象；第 9～16 行代码定义测试方法 sendSimpleMailTest()用于测试纯文本内容的邮件发送，其中第 11～13 行代码依次设置邮件发送的收件人、邮件的标题、邮件的正文内容，第 15 行代码根据设置的邮件信息调用 SendEmailService 中的邮件发送方法。

5. 测试效果

启动文件 9-11 中的 sendSimpleMailTest()方法，控制台输出发送纯文本邮件的结果，具体如图 9-9 所示。

从图 9-9 可以看出，控制台输出“ 纯文本邮件发送成功 ”的提示信息，说明文件 9-11 中发送邮件的方法执行成功。登录收件人的邮箱，在收件箱中查看邮件，效果如图 9-10 所示。

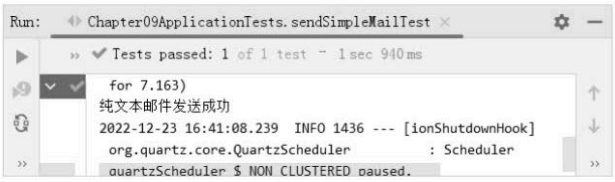


图9-9 发送纯文本邮件的结果



图9-10 收件人邮箱的收件箱的邮件

从图 9-10 可以看出，收件人邮箱的收件箱中成功收取到对应的纯文本邮件，邮件标题和邮件正文内容也与发送的信息一致，说明纯文本邮件业务实现成功。

9.2.2 发送带附件和图片的邮件

发送纯文本邮件任务的实现相对来说比较简单，但多数时候，可能需要在发送邮件的正文内容中嵌入图片等静态资源，或者在发送邮件的时候需要携带附件，针对上述需求，使用 SimpleMailMessage 对象封装邮件信息已经满足不了对应的需求，可以使用 JavaMailSenderImpl。JavaMailSenderImpl 除了可以发送 SimpleMailMessage 类型的邮件信息外，还可以发送 MimeMessage 类型的信息。MimeMessage 是指 MIME（Multipurpose Internet Mail Extensions，

多用途互联网邮件扩展）类型的邮件信息，是描述消息内容类型的因特网标准，能包含文本、图像、音频、视频以及其他应用程序专用的数据。

为了更好地屏蔽发送邮件的细节，Spring 提供了 MimeMessageHelper 类，MimeMessageHelper 类可以帮助构建 MimeMessage 对象中的内容。下面通过发送 MimeMessage 类型的消息，在 Spring Boot 项目中实现包含静态资源和附件的复杂邮件的发送。

1. 定义复杂邮件发送方法

在文件 9-10 的 SendEmailService 类中定义发送复杂邮件的方法 sendComplexEmail()，在该方法中通过 MimeMessageHelper 构建 MimeMessage 对象中的内容，实现发送正文内容包含图片并添加了附件的邮件，具体代码如下。

```
1  public void sendComplexEmail(String to,String subject,String text,
2         String filePath,String rscId,String rscPath){
3      // 定制复杂邮件信息
4      MimeMessage message = mailSender.createMimeMessage();
5      try {
6          // 使用 MimeMessageHelper 构建 MimeMessage 对象中的内容
7          MimeMessageHelper helper = new MimeMessageHelper(message, true);
8          helper.setFrom(from);
9          helper.setTo(to);
10         helper.setSubject(subject);
11         helper.setText(text, true);
12         FileSystemResource res = new FileSystemResource(new File(rscPath));
13         // 添加邮件静态资源
14         helper.addInline(rscId, res);
15         FileSystemResource file = new FileSystemResource(new File(filePath));
16         String                          fileName                          =
filePath.substring(filePath.lastIndexOf(File.separator));
17         // 添加邮件附件
18         helper.addAttachment(fileName, file);
19         // 发送邮件
20         mailSender.send(message);
21         System.out.println("复杂邮件发送成功");
22     } catch (MessagingException e) {
23         System.out.println("复杂邮件发送失败 "+e.getMessage());
24         e.printStackTrace();
25     }
26 }
```

在上述代码中，第 4 行代码通过 JavaMailSenderImpl 对象创建了用于封装复杂邮件信息的 MimeMessage 对象。第 7～18 行代码创建了 MimeMessageHelper 对象，并通过该对象构建了 MimeMessage 类中的内容，其中，第 8～11 行代码与发送纯文本邮件的代码一样，依次设置的是邮件的发件人地址、收件人地址、邮件标题、邮件正文；第 12～14 行代码读取文件系统下的资源；并将该资源设置为邮件内嵌静态资源；第 15～18 行代码读取文件系统下的资源，并将该资源设置为邮件附件。第 20 行代码用于发送设置好内容的邮件信息。

2. 定义测试方法

在文件 9-11 的测试类 Chapter09ApplicationTests 中新增测试复杂邮件发送的方法，具体如下。

```
@Test
public void sendComplexEmailTest() {
    String to="1175559518@qq.com";
    String subject="【复杂邮件】";
    // 定义邮件正文内容
    StringBuilder text = new StringBuilder();
    text.append("<html><head></head>");
    text.append("<body><h1>祝大家元旦快乐！</h1>");
    // cid 为固定写法，rscId 自定义的资源唯一标识
    String rscId = "img001";
    text.append("<img src='cid:" +rscId+"'/></body>");
    text.append("</html>");
    // 指定静态资源文件和附件路径
    String rscPath="E:\\email\\newyear.png";
    String filePath="E:\\email\\元旦放假注意事项.txt";
    // 发送复杂邮件
    sendEmailService.sendComplexEmail(to,subject,text.toString(),
            filePath,rscId,rscPath);
}
```

在上述代码中，根据前面定义的复杂邮件发送业务方法定制了各种参数。其中，在定义邮件内容时使用了 HTML 标签编辑邮件内容，内嵌了一个标识为 rscId 的图片，并为邮件指定了携带的附件路径。在邮件发送之前，应保证指定路径下存放有对应的静态资源和附件文件。

需要说明的是，编写内嵌静态资源文件时，cid 为嵌入式静态资源文件关键字的固定写法，如果改变将无法识别；rscId 则属于自定义的静态资源唯一标识，一个邮件内容中可能会包括多个静态资源，该属性是为了对这些静态资源加以区别。

3. 测试效果

启动文件 9-11 中的 sendComplexEmailTest()方法，控制台输出发送复杂邮件的结果，如图 9-11 所示。

从图 9-11 可以看出，控制台输出“复杂邮件发送成功”的提示信息，说明文件 9-11 中发送复杂邮件的方法执行成功。登录收件人的邮箱，在收件箱中查看邮件，效果如图 9-12 所示。

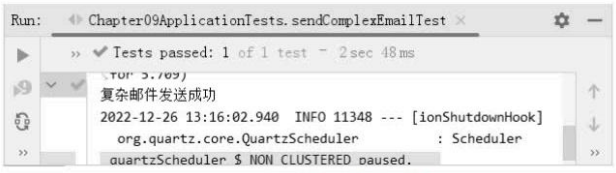


图9-11 发送复杂邮件的结果



图9-12 收件人邮箱的收件箱的复杂邮件

从图 9-12 可以看出，指定的收件人邮箱正确接收到了定制的复杂邮件，该复杂邮件包括一张内嵌在邮件内容中的静态资源图片和一个附件文件，这说明前面编写的带附件和图片的邮件业务实现成功。

9.2.3　发送模板邮件

前面两个案例中，分别对纯文本邮件和带附件及图片的复杂邮件的使用进行了讲解和实现，掌握这两种邮件发送可以完成开发中大部分的邮件发送任务。但是这两种邮件的实现必须每次都手动定制邮件内容，这在一些特定邮件发送任务中是相当麻烦的，例如用户注册验证邮件等，这些邮件的主体内容基本一样，主要是一些用户名、验证码、激活码等有所不同，所以，针对类似这种需求，可以定制一些通用邮件模板进行邮件发送。下面在 Spring Boot 项目中实现这种模板邮件的发送。

1. 添加 Thymeleaf 模板引擎依赖

使用定制邮件模板的方式实现通用邮件的发送，少不了前端模板页面的支持，这里选择使用 Thymeleaf 模板引擎定制模板邮件内容。在项目 chapter09 的 pom.xml 文件中添加 Spring Boot 整合 Thymeleaf 模板引擎的依赖启动器，具体代码如下。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

2. 定制邮件模板文件

在项目的模板页面文件夹 templates 中添加用户注册验证码的邮件的模板文件，内容如文件 9-12 所示。

文件 9-12　emailTemplate_vercode.html

```
<!DOCTYPE html>
<html lang="zh" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <title>用户验证码</title>
</head>
<body>
    <div><span th:text="${username}">XXX</span> 先生/女士,您好：</div>
    <P style="text-indent: 2em">您的新用户验证码为<span th:text="${code}"
                        style="color: cornflowerblue">123456</span>，请妥善保管。
</P>
</body>
</html>
```

在文件 9-12 中，主要模拟给注册用户发送一个动态验证码的邮件页面，从内容可以看出，该模板页面上包含两个变量，分别是用户名 username 和验证码 code，这两个变量在后续邮件发送后进行动态填充。

3. 定义邮件发送方法

在文件 9-10 的 SendEmailService 类中定义发送验证码邮件的方法 sendTemplateEmail()，在该方法中实现发送 HTML 模板邮件，具体代码如下。

```
public void sendTemplateEmail(String to, String subject, String content) {
    MimeMessage message = mailSender.createMimeMessage();
```

```
3      try {
4          // 使用 MimeMessageHelper 设置 MimeMessage 的内容
5          MimeMessageHelper helper = new MimeMessageHelper(message, true);
6          helper.setFrom(from);
7          helper.setTo(to);
8          helper.setSubject(subject);
9          helper.setText(content, true);
10         // 发送邮件
11         mailSender.send(message);
12         System.out.println("模板邮件发送成功");
13     } catch (MessagingException e) {
14         System.out.println("模板邮件发送失败 "+e.getMessage());
15         e.printStackTrace();
16     }
17 }
```

在上述代码中，sendTemplateEmail()方法与之前发送邮件的逻辑类似，将邮件的内容设置在 MimeMessage 对象中进行发送，不同之处在于第 9 行代码调用 setText()方法设置的文本内容作为解析后的模板页面的内容。

4. 定义测试方法

在项目 chapter09 测试类 Chapter09ApplicationTests 中新增测试模板邮件发送的方法，具体如下。

```
1  @Autowired
2  private TemplateEngine templateEngine;
3  @Test
4  public void sendTemplateEmailTest() {
5      String to="1175559518@qq.com";
6      String subject="【验证码邮件】";
7      // 使用模板邮件定制邮件正文内容
8      Context context = new Context();
9      context.setVariable("username", "黑马");
10     context.setVariable("code", "456123");
11     // 使用 TemplateEngine 设置要处理的模板页面
12     String emailContent = templateEngine.process("emailTemplate_vercode",
context);
13     // 发送模板邮件
14     sendEmailService.sendTemplateEmail(to,subject,emailContent);
15 }
```

在上述代码中，第 1～2 行代码注入 Thymeleaf 提供的模板引擎解析器 TemplateEngine。在第 4～15 行代码的测试方法中设置发送模板邮件所需的参数后调用邮件发送方法。其中，第 8～12 行代码使用 Context 对象对模板邮件中涉及的变量 username 和 code 动态赋值，然后使用模板引擎解析器的 process(String template, IContext context)方法对模板文件进行解析，该方法的第一个参数是要解析的 Thymeleaf 模板页面，第二个参数为设置页面中的动态数据。

5. 测试效果

启动文件 9-11 中的 sendTemplateEmailTest()方法，登录收件人的邮箱，在收件箱中查看验证码邮件，效果如图 9-13 所示。



图9-13　验证码邮件

从图 9-13 可以看出，指定的收件人邮箱正确接收到了定制的验证码邮件，并且该验证码邮件中涉及的两个变量 username 和 code 都被动态赋值，说明了前面编写的模板邮件业务实现成功。

至此，关于 Spring Boot 对邮件发送的支持已经讲解完毕了。

9.3　本章小结

本章主要对 Spring Boot 项目中的任务调度和邮件发送进行了讲解。首先讲解了任务调度中的异步任务、定时任务，以及 Quartz 任务调度；然后讲解了邮件发送，包括发送纯文本邮件、发送带附件和图片的邮件，以及发送模板邮件。通过本章的学习，希望读者可以在 Spring Boot 项目中实现任务调度和邮件发送。

9.4　本章习题

一、填空题

1．被_____注解标注的方法称之为异步方法。

2．Spring Boot 中处理有返回值的异步方法中，返回值的类型须为_____类型。

3．_____注解是 Spring 框架提供的定时任务控制的注解。

4．JavaMailSenderImpl 对象的_____方法用于执行邮件发送。

5．发送模板邮件时，需要使用模板引擎解析器的 process()方法对模板文件进行_____。

二、判断题

1．异步方法在执行的时候，会在独立的线程中执行。(　　)

2．fixedDelay 和 fixedDelayString 属性的下一次执行时间是在上一次任务执行完成后开始计时。(　　)

3．使用@Scheduled 注解的 fixedRate 属性时可能会引起线程的阻塞。(　　)

4．@Scheduled 注解 fixedDelayString 属性作用与 fixedDelay 属性相同，不过属性值为字符串形式的数值。(　　)

5．发送模板邮件时，MimeMessageHelper 执行 setText()方法设置的文本内容为解析前的模板页面的内容。(　　)

三、选择题

1．下列选项中，用于开启基于注解的异步任务支持的注解是(　　)。

A．@Async　　B．@EnableAsync　　C．@Controller　　D．@EnableWebMvc

2．下列选项中，关于@EnableScheduling 注解的作用描述正确的是（　　）。

A．标注执行异步任务方法的注解。

B．用于开启基于注解方式的定时任务支持。

C．开启基于注解的异步任务支持。

D．用于定时任务控制的注解。

3．下列选项中，对@Scheduled(fixedDelay = 5000)的作用描述正确的是（　　）。

A．在程序启动后，会立即执行一次定时任务，在任务执行结束后，每隔 5000 毫秒重复执行一次任务。

B．在程序启动后，每隔 5000 毫秒重复执行一次任务。

C．程序启动后，延迟 5000 毫秒执行第一次任务。

D．在任务执行结束后，等待 5000 毫秒，然后再执行一次任务；但只触发一次定时任务。

4．下列选项中，Quartz 中调度程序执行的任务组件是（　　）。

A．Job　　B．JobDetai　　C．Trigger　　D．Scheduler

5．下列选项中，对 MimeMessageHelper 的 addInline()方法描述正确的是（　　）。

A．添加邮件静态资源。

B．添加邮件附件。

C．设置邮件正文内容。

D．设置邮件发件人地址。

第 10 章

Spring Boot综合项目实战——瑞吉外卖

学习目标

★ 了解系统概述，能说出系统包含的主要功能和技术

★ 掌握开发环境搭建，能够基于系统开发及运行环境搭建数据库环境和项目环境

★ 掌握管理端功能模块，能够实现管理端的登录管理、分类管理、菜品管理、套餐管理、订单明细功能模块

★ 掌握用户端功能模块，能够根据提供的用户端代码实现用户登录、地址管理、菜品展示、购物车、下单功能模块

通过前面的学习，读者应该已掌握了 Spring Boot 框架，以及其他常用技术的基本知识和整合使用，通过这些已学的相关知识，已经可以在实际工作中进行基本的项目开发。下面使用前面已学的 Spring Boot 相关知识，以及其他常用的技术实现一个外卖管理系统的开发，使读者可更熟练地掌握 Spring Boot 框架及相关技术的使用。

10.1 系统概述

10.1.1 系统功能介绍和技术选型

外卖管理系统主要用于实现外卖订购、处理、配送等各个环节的流程自动化。在设计外卖管理系统时，除了关注系统功能和性能外，还需要将社会责任的理念贯穿到系统的每个功能模块中，在设计和实现功能时要遵循职业道德的准则，例如诚实守信、尊重他人的知识产权和隐私、遵守法律法规等，以保护用户利益和权益，维护行业形象和信誉。同时，应履行维护社会稳定、经济发展、人民幸福的社会责任，通过科技创新为用户提供更好的服务，体现科技与社会的有机联系。

1. 系统功能

本系统是为餐厅、饭店等餐饮企业定制的一款软件产品，包括管理端和用户端两个部分，其中管理端为系统管理后台，主要提供给餐饮企业内部员工使用，可以对餐厅的分类、菜品、套餐、订单等信息进行管理维护。用户端主要提供给消费者使用，可以在线浏览菜品、添加购物车、下单等。下面通过一张图展示本项目的系统功能，如图 10-1 所示。

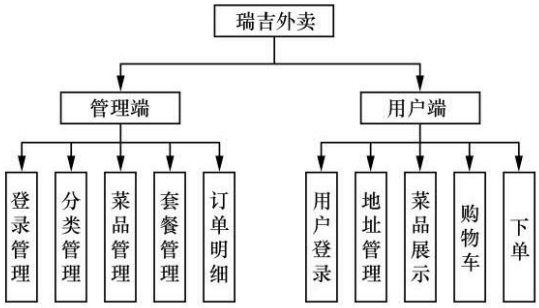


图10-1 系统功能架构

2. 技术选型

Spring Boot 致力于提高基于 Spring 体系的 Java EE 企业级开发速度，是现代 Java EE 开发中的必选技术。通常企业级应用的实现相对复杂，需要多种技术共同实现应用中的业务，本章将使用 Spring Boot 集成其他常用主流框架，以实现一个外卖管理系统。下面通过一张图展示本系统使用的核心技术，如图 10-2 所示。

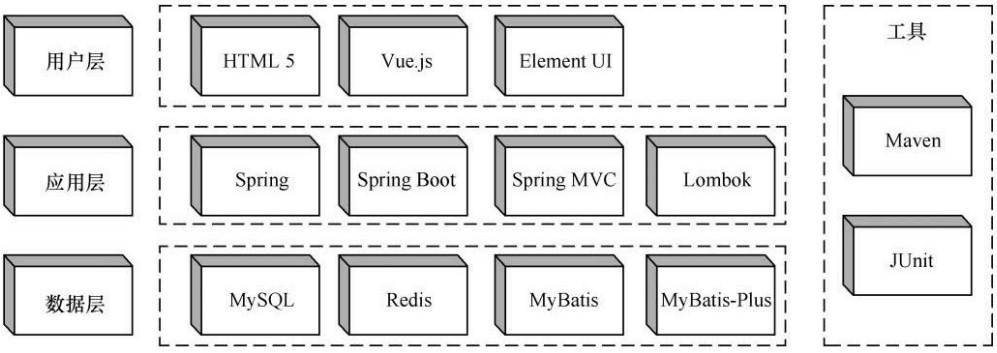


图10-2 系统使用的核心技术

图 10-2 中，根据技术在系统的使用场景分为用户层、应用层、数据层和工具四个部分，下面分别对这些技术进行说明。

- 用户层：使用 HTML5、Vue.js 和 Element UI 构建系统页面。
- 应用层：Spring 统一管理项目中的 Bean，Spring MVC 和 Spring 无须通过中间整合层进行整合，可以无缝集成。Spring Boot 可以快速构建 Spring 项目，采用“约定优于配置”的思想，简化了 Spring 项目的配置开发。Lombok 是一个 Java 库，它可以自动插入编辑器和构建工具中，能以简单的注解形式来简化 Java 代码，提高开发人员的开发效率。例如，可避免编写 Java Bean 时添加相应的 getter/setter 方法，以及构造器、equals()等方法。
- 数据层：MySQL 为关系型数据库，本系统的核心业务数据都会采用 MySQL 进行存储。Redis 基于 Key-Value 格式存储的内存数据库，访问速度快，本系统中使用 Redis 实现缓存，可降低数据库访问压力，提高访问效率。本系统的持久层将使用 MyBatis-Plus 来简化开发，基本的单表增删改查可直接调用框架提供的方法。
- 工具：本系统使用 Maven 作为项目的构建工具，并使用 JUnit 作为系统的单元测试工具，开发人员实现好功能后，可以通过 JUnit 对功能进行单元测试。

10.1.2　项目效果预览

为了使读者对本章要讲解的外卖管理系统有整体、直观的认识，下面结合图 10-2 中的系统功能展示项目主要功能页面的预览效果。

1. 登录管理

员工必须成功登录系统后才可以访问系统管理后台，本项目员工登录页面预览效果如图 10-3 所示。

2. 分类管理

分类管理主要对当前餐厅经营的菜品和套餐进行分类管理维护， 包括查询、新增、修改、删除分类等功能。本项目的分类管理页面预览效果如图 10-4 所示。



图10-3　员工登录页面预览效果

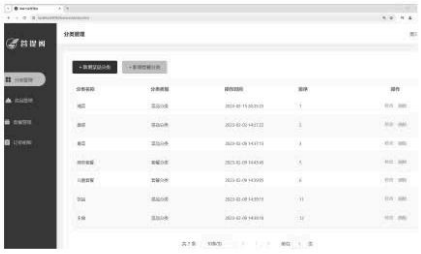


图10-4　分类管理页面预览效果

3. 菜品管理

菜品管理主要维护各个分类下的菜品信息，包括查询、新增、修改、删除、启售、停售等功能。本项目的菜品管理页面预览效果如图 10-5 所示。

4. 套餐管理

套餐管理主要维护当前餐厅中的套餐信息，包括查询、新增、修改、删除、启售、停售等功能。本项目的套餐管理页面预览效果如图 10-6 所示。

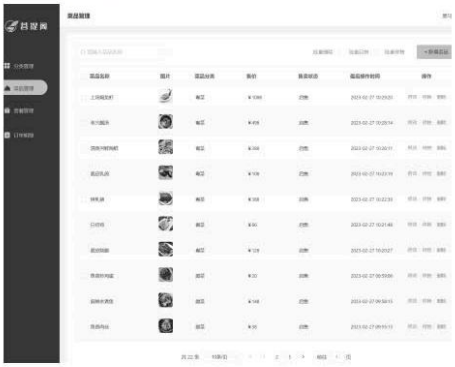


图10-5　菜品管理页面预览效果



图10-6　套餐管理页面预览效果

5. 订单明细

订单明细主要维护用户在移动端下的订单信息，包括查询、取消、派送、完成等功能。本项目的订单明细页面预览效果如图 10-7 所示。

6. 用户登录

用户登录是用户端的功能，在用户端用户需要登录后才可以使用其他功能。本项目的用户登录页面预览效果如图 10-8 所示。



图10-7 订单明细页面预览效果



图10-8 用户登录页面预览效果

7. 地址管理

用户登录成功后可以维护自己的地址信息。同一个用户可以有多个地址信息，但是只能有一个默认地址。地址管理包括对地址的新增、查询、编辑、删除操作，以及设置默认地址。本项目的地址管理页面预览效果如图 10-9 所示。

8. 菜品展示

用户登录成功后跳转到用户的首页，用户端首页根据分类来展示菜品和套餐。本项目的菜品展示页面预览效果如图 10-10 所示。



图10-9 地址管理页面预览效果



图10-10 菜品展示页面预览效果

9. 购物车

用户选中的菜品就会加入用户的购物车，购物车功能主要包括查询购物车、加入购物车、删除购物车、清空购物车等功能。本项目的购物车预览效果如图 10-11 所示。

10. 下单

用户将加入购物车的菜品提交结算请求并确认订单，对订单进行支付后完成下单操作。

本项目的下单页面预览效果如图 10-12 所示。



图10-11　购物车预览效果



图10-12　下单页面预览效果

通过图 10-3～图 10-12 展示了餐厅外卖系统主要页面的预览效果，后续讲解中将对这些功能逐个加以实现。

10.2　开发环境搭建

10.2.1　系统开发及运行环境

为了使读者能更方便地学习本项目的开发，避免学习过程中出现错误，下面对本系统开发及运行所需的环境和相关软件进行介绍，具体如下。

- 操作系统：Windows 10。
- Java 开发包：JDK 11。
- 项目管理工具：Maven 3.6.3。
- 项目开发工具：IntelliJ IDEA 2022.2.2。
- 数据库：MySQL 8.0。
- 缓存管理工具：Redis 5.0.14.1。
- 浏览器：Google Chrome。

上述软件或工具，读者在学习时可以自行从网上下载，本书配套的资源中也会提供，读者可以自行选择。

10.2.2　数据库环境搭建

在程序的实现过程中，需要不断增强信息安全意识，建立完善的系统权限管理模块，确保系统的安全性和用户的隐私不会受到损害，避免恶意攻击和滥用。同时也应尊重用户隐私权、拒绝侵犯用户权益、增强维护公共利益的意识，为维护健康的网络环境贡献自己的一份力量。

本项目涉及的数据表相对较多，为了方便读者使用对应的数据表，本书的配套资源中提供了对应的 SQL 文件。首先创建一个名称为 reggie 的数据库，创建好该数据库后，将本书资源中所提供的 db_reggie.sql 文件导入到 reggie 数据库中。导入的 SQL 文件中创建的数据表如表 10-1 所示。

表 10-1　创建的数据表

序号	表名	说明
1	employee	员工表
2	category	菜品和套餐分类表
3	dish	菜品表
4	setmeal	套餐表
5	setmeal_dish	套餐菜品关系表
6	dish_flavor	菜品口味关系表
7	user	用户表
8	address_book	地址簿表
9	shopping_cart	购物车表
10	orders	订单表
11	order_detail	订单明细表

需要说明的是，本书提供的 db_reggie.sql 文件中，除了包含创建数据表的语句外，也包含一些数据插入语句，导入 db_reggie.sql 文件后部分数据表中会包含一些基本的初始化数据，可供读者测试使用。

10.2.3　项目环境搭建

本项目以 Maven 作为项目的构建工具，下面依次执行创建项目和配置依赖、编写配置文件、导入前端资源、导入后端基础文件等操作，完成项目环境的搭建，具体如下。

1. 创建项目和配置依赖

在 IDEA 中创建一个名称为 reggie 的 Maven 项目，项目创建完成后，在项目的 pom.xml 文件中配置项目所需的依赖，具体如文件 10-1 所示。

文件 10-1　pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.7.6</version>
        <relativePath/>
    </parent>
    <groupId>com.itheima</groupId>
```

```
    <artifactId>reggie</artifactId>
    <version>1.0-SNAPSHOT</version>
    <properties>
        <java.version>1.11</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>com.baomidou</groupId>
            <artifactId>mybatis-plus-boot-starter</artifactId>
            <version>3.4.2</version>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>1.18.20</version>
        </dependency>
        <dependency>
            <groupId>com.alibaba</groupId>
            <artifactId>fastjson</artifactId>
            <version>1.2.76</version>
        </dependency>
        <dependency>
            <groupId>commons-lang</groupId>
            <artifactId>commons-lang</artifactId>
            <version>2.6</version>
        </dependency>
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>com.alibaba</groupId>
            <artifactId>druid-spring-boot-starter</artifactId>
            <version>1.1.23</version>
        </dependency>
        <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-redis</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-cache</artifactId>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <version>2.7.6</version>
            </plugin>
        </plugins>
    </build>
</project>
```

上述展示的本项目所需依赖主要包括 Spring Boot 启动器、Spring Web 场景启动器、MyBatis-Plus 整合 Spring Boot 的启动器、数据库连接等。

2. 编写配置文件

在项目的 resources 文件夹下创建配置文件 application.yml ,并在该配置文件中配置服务端口号、MySQL 数据库连接信息、MyBatis-Plus 配置、常量配置等信息，具体如文件 10-2 所示。

文件 10-2 application.yml

```
server:
  port: 8080
spring:
  application:
    #应用的名称，可选
    name: reggie
# 数据源配置
  datasource:
    druid:
      driver-class-name: com.mysql.cj.jdbc.Driver
      url: "jdbc:mysql://localhost:3306/reggie?serverTimezone=Asia/Shanghai

&useUnicode=true&characterEncoding=utf-8&zeroDateTimeBehavior=convertToNull
      &useSSL=false&allowPublicKeyRetrieval=true"
      username: root
      password: root
# MyBatis-Plus 配置
mybatis-plus:
  configuration:
    #在映射实体或者属性时，将数据库中表名和字段名中的下画线去掉，按照驼峰命名法映射
    map-underscore-to-camel-case: true
    # SQL 语句等日志信息输出在控制台
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl
  global-config:
```

```
24    db-config:
25      # 使用雪花算法自动生成主键 ID
26      id-type: ASSIGN_ID
27 # 常量配置，存放图片的目录
28 reggie:
29   path: D:\img\
```

3. 导入前端资源

本项目主要根据项目的业务要求实现后台功能代码，前端页面读者可直接将本书提供的配套资源导入项目，在功能实现时会对页面中已经实现的逻辑进行讲解。在将前端资源导入到项目的 resources 文件夹后，resources 文件夹的目录结构如图 10-13 所示。

从 10.1 节系统概述的讲解中可知，本项目分为管理端和用户端，图 10-13 中的 backend 文件夹下是管理端对应的前端静态资源，front 文件夹下是用户端对应的前端静态资源，这两个文件夹下的 page 文件夹为管理端和用户端中各功能模块对应的页面，其他文件夹分别为页面所引用的 Java Script 文件、图片、CSS 样式等静态资源。

4. 导入后端基础文件

在正式开发后端业务之前，为了简化后续业务开发，本书配套资源中提供了一些非核心功能的基础文件，这些文件包括通用类、配置类、实体类和工具类等，将这些基础文件导入项目的 src/main/目录的 java 文件夹下，导入后 java 文件夹下的目录结构如图 10-14 所示。



图10-13　resources文件夹的目录结构

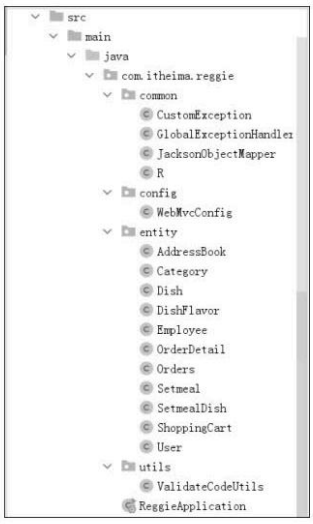


图10-14　java文件夹下的目录结构

在图 10-14 中将通用类、配置类、实体类和工具类存放在不同的包中，其中 common 包下依次为自定义业务异常类 CustomException、全局异常处理类 GlobalExceptionHandler、将 JSON 和 Java 对象相互转换的 JSON 对象映射器 JacksonObjectMapper、通用返回结果类 R；config 包下包含 Spring MVC 的配置类 WebMvcConfig；entity 包下为项目中涉及的所有实体类；utils 包下为生成随机验证码的工具类 ValidateCodeUtils。

entity 包下的实体类使用 Lombok 的注解省去了手动编写类对应的 setter/getter 方法、构造方法、toString()方法等。想要使用 Lombok，除了需要在 pom.xml 文件中配置对应的依赖外，还需要在 IDEA 中安装一个 Lombok 插件，否则 IDEA 会无法识别 Lombok 的注解。依

次单击 IDEA 菜单中的“File”→“Settings”进入到“Settings”对话框，在“Settings”对话框中选中直接在工具中搜索 Lombok 插件，然后安装它。安装完成后重启 IDEA。

至此，瑞吉外卖管理系统的项目环境已经搭建完成。

10.3 管理端功能模块

管理端包括登录管理、分类管理、菜品管理、套餐管理和订单明细等功能，这些功能只面向餐厅内部人员，餐厅内部人员通过合理的管理为用户端提供资源和数据。下面依次对管理端的登录管理、分类管理、菜品管理、套餐管理和订单明细的功能进行讲解。

10.3.1 登录管理



登录管理包括员工登录和员工退出，员工在登录页面成功登录后，会进入到后台管理系统，拥有后台系统中的所有操作权限。当员工退出后清除登录状态，返回到登录页面。

登录管理的具体实现请扫描右侧二维码查看。

10.3.2 分类管理



管理端中的分类管理是对菜品和套餐的分类信息进行管理，每个菜品对应一个菜品分类，每个套餐对应一个套餐分类，用户端中会按照菜品和套餐的分类信息展示不同分类的菜品信息和套餐信息。

分类管理的具体实现请扫描右侧二维码查看。

10.3.3 菜品管理



菜品是餐厅的核心内容，消费者在用户端主要通过浏览菜品信息进行购买。菜品管理主要维护各个分类下的菜品信息，包括新增、查询、修改、启售、停售、删除等功能。

菜品管理的具体实现请扫描右侧二维码查看。

10.3.4 套餐管理



套餐是多个菜品的集合，通过合理的套餐设置，能够一次性满足消费者的点餐需求，帮助消费者快速下单。套餐管理主要维护所有套餐分类的套餐信息，包括新增、查询、修改、启售、停售、删除等功能。

套餐管理的具体实现请扫描右侧二维码查看。

10.3.5 订单明细



订单明细可以记录用户下单后的数据，通过订单明细页面可以进行查询订单和修改订单状态等操作，是后台系统中较为重要的功能之一。

订单明细的具体实现请扫描右侧二维码查看。

10.4　用户端功能模块

用户端用于展示管理端设置好的菜品和套餐，消费者可根据需求将菜品或套餐添加到购物车并下单。本系统用户端的功能基本也是对单表的增删改查，与管理端的开发思路基本一样。因此，用户端功能模块的功能代码就不再一一实现了，基本的代码在配套的资源中都已经提供，直接导入到项目中即可。下面依次对用户端的用户登录、地址管理、菜品展示、购物车、下单的实现逻辑和功能测试进行讲解。

10.4.1　用户登录

为了方便用户登录，当前比较流行的方式是基于短信验证码进行登录。想要实现短信发送功能，无须自己实现，也无须与运营商直接对接，只需要调用第三方提供的短信服务即可。目前市面上有很多第三方提供的短信服务，这些第三方短信服务商会与各个运营商对接，只需要注册成为会员，并且按照提供的开发文档进行调用就可以发送短信。需要说明的是，这些短信服务一般都是收费服务。

本项目中发送验证码的功能并不是真正发送获取验证码短信的请求，而是选择通过工具类模拟发送和获取验证码。工具类中提供方法生成验证码信息，并通过日志输出，登录时直接从控制台中获取生成的验证码。由此可知，为实现用户登录功能，前端页面需要发送获取验证码和用户登录两次请求。


10.4.1

用户登录的具体讲解请扫描右侧二维码查看。

10.4.2　地址管理

地址是指用户的地址信息，即收取外卖的地址。用户登录成功后可以维护自己的地址信息，地址信息中可以设置默认地址，同一个用户可以有多个地址信息，但是只能设置一个默认地址。地址管理包括地址列表查询、新增地址、设置默认地址、编辑地址、删除地址。


10.4.2

地址管理的具体讲解请扫描右侧二维码查看。

10.4.3　菜品展示

用户登录成功后会跳转到用户端的首页，首页展示所有分类，以及被选中的分类所包含的具体菜品。


10.4.3

菜品展示的具体讲解请扫描右侧二维码查看。

10.4.4　购物车

在用户端用户可以将菜品或者套餐添加到购物车。对于菜品来说，如果设置了口味信息，则需要选择规格后才能加入购物车；对于套餐来说，可以直接将当前套餐加入购物车。在购物车中可以修改菜品和套餐的数量，也可以清空购物车。因此，可以将购物车拆分为添加购物车、查询购物车、修改购物车这 3 个功能。


10.4.4

购物车的具体讲解请扫描右侧二维码查看。

10.4.5 下单

下单是指用户对购物车中的商品确认无误后，提交结算请求，系统会生成订单，用户对该订单进行支付的整个过程。

下单的具体讲解请扫描右侧二维码查看。

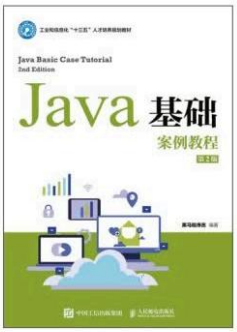

10.4.5

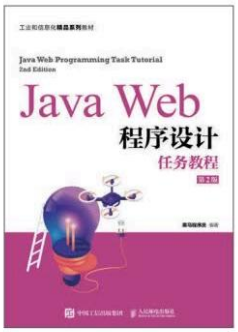
10.5 本章小结

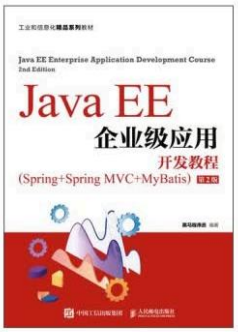
本章主要围绕 Spring Boot 框架，整合相关技术实现了一个外卖管理系统。通过本章的学习，读者可以更加熟悉项目系统的架构设计和相关组织结构，同时能够掌握 Spring Boot 项目的环境搭建 ,最重要的是能够学习 Spring Boot 框架整合其他技术进行业务的开发实现。在本章项目学习过程中，读者务必要动手实践体验 Spring Boot 综合项目的开发流程。

应用型IT套系教材推荐

RECOMMEND APPLICATION IT SERIES TEXTBOOK

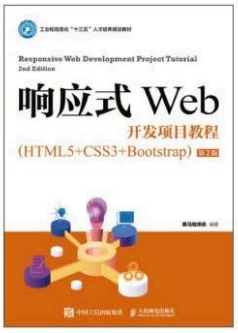




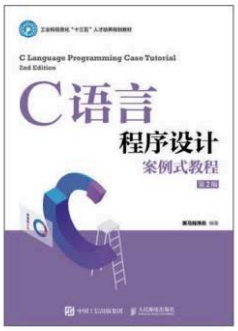


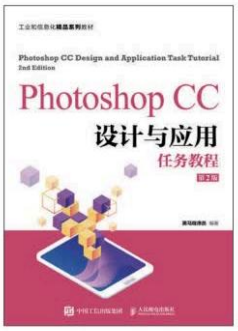












扫此二维码下载
本书配套资源

人邮教育
www.ryjiaoyu.com

教材服务热线：010-81055256
反馈 / 投稿 / 推荐信箱：315@ptpress.com.cn
人邮教育服务与资源下载社区：www.ryjiaoyu.com

ISBN 978-7-115-63438-2